Folleto del estudiante Encuentro 1: Aplicaciones de interfaz gráfica

Departamento de Ciencias, Tecnología y Salud – CUR Matagalpa

Fecha: ___/___/2026

UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA, MANAGUA
UNAN-MANAGUA

UNIVERSIDAD EN EL campo

Universidad Nacional Autónoma de Nicaragua, Managua · Centro Universitario Regional de Matagalpa · Departamento de Ciencias, Tecnología y Salud

Programación de Aplicaciones de Escritorio – Folleto del Encuentro 1: Aplicaciones de interfaz gráfica

Heydy Judelki Mairena Palacios

Ingeniería en Sistemas · II año – II semestre 2026 · Unidad(es) I del programa

Fecha de emisión: ___/___/2026

¡Universidad del Pueblo y para el Pueblo!

Elaborado por:	Revisado por:	Autorizado por:
Heydy Judelki Mairena Palacios	⚠ PENDIENTE (coordinador/a de carrera)	⚠ PENDIENTE (director/a de departamento)
Perfil del autor (opcional): ⚠ completar si se desea.		

Presentación

Este primer módulo inicia la construcción de AgroStock, la aplicación de escritorio que desarrollaremos durante el semestre para el inventario de un agroservicio. Aprenderás cómo está formada una aplicación de interfaz gráfica en C#, cómo se colocan y configuran controles, cómo el programa responde a las acciones del usuario mediante eventos, y cómo se organizan menús, barras de herramientas y cajas de diálogo. Al final del encuentro tendrás la versión 1 de AgroStock funcionando.

Proyecto integrador: AgroStock: aplicación de escritorio para el inventario de un agroservicio o cooperativa de la zona (productos, categorías, existencias, reportes y tipo de cambio). Se construye por etapas: v1 interfaz gráfica (E1) → v2 base de datos SQLite + importación concurrente (E2) → v3 ORM, reportes PDF/Excel y consumo de servicio web (E3).

Horas del programa para este módulo: HT 10 · HP 20 · HTI 15 (total 45 h).

Competencias y resultados de aprendizaje

Competencias del programa a las que contribuye este módulo:

- Desarrolla aplicaciones de escritorio con interfaz gráfica que acceden a bases de datos, implementan programación concurrente (hilos), utilizan ORM, generan reportes y consumen servicios web; mediante la aplicación de un lenguaje de programación orientado a objetos para resolver problemas de la vida cotidiana.
- Aplica procesos creativos e investigativos haciendo uso de medios tecnológicos en el desarrollo de sus aprendizajes.
- Demuestra actitudes de respeto hacia las personas y los saberes ancestrales, inclusión y equidad para una cultura de paz en su entorno.

Resultados de aprendizaje:

- Desarrolla aplicaciones de escritorio con interfaz gráfica haciendo uso de controles, manejo de eventos y cajas de diálogo.

Objetivos del encuentro:

- Explicar la estructura de una aplicación de escritorio con interfaz gráfica en C# (Windows Forms): punto de entrada, formularios, controles y ciclo de eventos.
- Construir formularios con controles, manejo de eventos, menús, barras de herramientas y cajas de diálogo para resolver un problema de inventario de un agroservicio local.

Contenidos

Unidad I. Aplicaciones de interfaz gráfica: Estructura de una aplicación de interfaz gráfica; Uso de controles en un formulario; Manejo de eventos; Menús y barras de herramientas; Cajas de diálogo.

I. Entrada temática – Vivencia

Doña Martha atiende un agroservicio en la salida hacia San Ramón. Anota en un cuaderno cada quintal de fertilizante que entra y sale. Cuando un productor pregunta si hay fórmula 18-46-0, ella revisa varias páginas; a fin de mes suma a mano y casi nunca coincide con lo que hay en bodega.

Preguntas generadoras (responde en tu cuaderno y comenta con tu pareja):

1. ¿Qué información anota doña Martha de cada producto? Haz una lista de campos.
2. Si tuvieras que diseñar la pantalla para registrar un producto, ¿qué control usarías para cada campo: cuadro de texto, lista desplegable, número, fecha?
3. ¿Qué debería pasar cuando ella pulsa «Guardar» y olvidó escribir el precio?
4. ¿Qué programas de escritorio conoces en tu comunidad (facturación, planillas, control de ventas)? ¿Qué tienen en común sus ventanas?

> Lo que esperamos aprender: al terminar este módulo podrás construir una aplicación de escritorio con ventanas, controles, eventos, menús y cajas de diálogo, y la aplicarás al caso de doña Martha.

II. Desarrollo central – Fundamentación científica

1. Estructura de una aplicación de interfaz gráfica

Una aplicación de interfaz gráfica de usuario (GUI) presenta la información en ventanas y responde a lo que el usuario hace con el ratón y el teclado. A diferencia de un programa de consola, que ejecuta instrucciones de arriba hacia abajo, una aplicación GUI es dirigida por eventos: después de mostrar la ventana, espera a que ocurra algo (un clic, una tecla, un temporizador) y ejecuta el código asociado (Ceballos, s. f.; Deitel y Deitel, s. f.).

En .NET, la tecnología Windows Forms permite construir estas aplicaciones con el lenguaje C#. Un proyecto Windows Forms típico contiene:

Archivo	Función
AgroStock.csproj	Configuración del proyecto: versión de .NET (net8.0-windows), uso de Windows Forms y paquetes NuGet.
Program.cs	Punto de entrada. El método Main inicializa la aplicación y llama a Application.Run(new FrmPrincipal()).
FrmPrincipal.cs	Clase del formulario que hereda de Form. Aquí se escribe el código de los eventos.
FrmPrincipal.Designer.cs	Código generado por el diseñador visual: crea los controles y asigna sus propiedades en InitializeComponent().

```
internal static class Program
{
    [STAThread]
    private static void Main()
    {
        ApplicationConfiguration.Initialize(); // estilos visuales y DPI
        Application.Run(new Forms.FrmPrincipal()); // inicia el bucle de mensajes
```

```
    }
}
```

Application.Run inicia el bucle de mensajes: el sistema operativo coloca en una cola cada acción del usuario y la aplicación la convierte en un evento del control correspondiente. El atributo [STAThread] indica que la interfaz se ejecuta en un único hilo (el hilo de la interfaz); lo retomaremos en la Unidad III.

Un formulario es una clase. Por eso aplicamos lo aprendido en Programación Orientada a Objetos: el formulario hereda de Form, sus controles son atributos privados y los manejadores de eventos son métodos.

2. Uso de controles en un formulario

Un control es un componente visual reutilizable. Cada control tiene propiedades (su estado: Text, Enabled, Visible), métodos (acciones: Focus(), Clear()) y eventos (avisos: Click, TextChanged). Elegir el control correcto evita errores de captura.

Control	Uso en AgroStock	Propiedad/evento clave
Label	Etiquetas de los campos	Text
TextBox	Código y nombre del producto	Text, MaxLength, PlaceholderText / TextChanged, KeyPress
ComboBox	Categoría (lista cerrada)	DropDownStyle = DropDownList, Items / SelectedIndexChanged
NumericUpDown	Precio y existencia	DecimalPlaces, Minimum, Maximum / ValueChanged
DateTimePicker	Fecha de vencimiento	Format, ShowCheckBox / ValueChanged
Button	Guardar, Cancelar	Text, DialogResult / Click
DataGridView	Lista de productos	DataSource, SelectionMode / CellDoubleClick

Para que la ventana se vea bien en cualquier tamaño de pantalla se usan las propiedades Dock (acoplar a un borde o llenar) y Anchor (mantener la distancia a los bordes), y contenedores como TableLayoutPanel (cuadrícula de filas y columnas) y FlowLayoutPanel (elementos uno tras otro).

```
var tabla = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 2 };
tabla.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 120)); // etiquetas
tabla.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100));  // controles
tabla.Controls.Add(new Label { Text = "&Nombre:", AutoSize = true });
tabla.Controls.Add(txtNombre);
```

> Convención de nombres: prefijo del tipo + propósito (txtNombre, cboCategoria, nudPrecio, btnGuardar, dgvProductos). Evita dejar button1 o textBox2: el código se vuelve ilegible.

3. Manejo de eventos

Un evento es un aviso que un objeto (el emisor) envía cuando algo ocurre. Otros objetos se suscriben con el operador += indicando el método que debe ejecutarse (el manejador). Todo manejador recibe dos parámetros: sender (quién generó el evento) y e (datos del evento, de tipo EventArgs o una clase derivada).

```
// Suscripción (en el constructor o en el diseñador)
btnGuardar.Click += BtnGuardar_Click;
txtCodigo.KeyPress += TxtCodigo_KeyPress;

// Manejador: solo letras, dígitos y guion; convierte a mayúscula
private void TxtCodigo_KeyPress(object? sender, KeyPressEventArgs e)
{
    if (char.IsControl(e.KeyChar)) return;          // permitir Retroceso
    if (!char.IsLetterOrDigit(e.KeyChar) && e.KeyChar != '-')
    {
        e.Handled = true;                           // se descarta la tecla
        return;
    }
    e.KeyChar = char.ToUpperInvariant(e.KeyChar);
}
```

Evento	¿Cuándo ocurre?	Ejemplo de uso
Load	Antes de mostrar el formulario por primera vez	Cargar datos iniciales
Click	Al pulsar un botón o elemento de menú	Guardar, eliminar
TextChanged	Cada vez que cambia el texto	Búsqueda mientras se escribe
KeyPress / KeyDown	Al pulsar una tecla	Filtrar caracteres, atajos (Ctrl+F)
CellDoubleClick	Doble clic en una celda del DataGridView	Abrir el producto para editar
FormClosing	Antes de cerrar la ventana (se puede cancelar)	Confirmar salida
Tick (Timer)	Cada intervalo del temporizador	Reloj en la barra de estado

La validación de datos se hace en el evento del botón Guardar (o en Validating de cada control). El componente ErrorProvider muestra un icono junto al control con el mensaje de error, sin interrumpir con ventanas emergentes.

```
private void BtnGuardar_Click(object? sender, EventArgs e)
{
    errores.Clear();
    if (txtNombre.Text.Trim().Length < 3)
    {
        errores.SetError(txtNombre, "Escriba al menos 3 caracteres.");
        return;                     // el diálogo sigue abierto
    }
    Producto.Nombre = txtNombre.Text.Trim();
    DialogResult = DialogResult.OK;    // cierra el formulario modal
}
```

4. Menús y barras de herramientas

Los menús organizan todas las funciones de la aplicación; las barras de herramientas ofrecen acceso rápido a las más usadas; la barra de estado informa sin interrumpir. En Windows Forms se usan MenuStrip, ToolStrip y StatusStrip, cuyos elementos (ToolStripMenuItem, ToolStripButton, ToolStripStatusLabel) comparten el mismo modelo de eventos.

- El carácter & antes de una letra crea la tecla de acceso: "&Archivo" se abre con Alt+A.
- ShortcutKeys asigna un atajo global: Keys.Control | Keys.N para Nuevo.
- ToolTipText muestra una ayuda al pasar el ratón sobre un botón.

- ContextMenuStrip crea el menú del clic derecho y se asigna a la propiedad ContextMenuStrip del control.

```
var archivo = new ToolStripMenuItem("&Archivo");
archivo.DropDownItems.Add(new ToolStripMenuItem("&Nuevo producto", null, (_, _) => Nuevo(),
Keys.Control | Keys.N));
archivo.DropDownItems.Add(new ToolStripSeparator());
archivo.DropDownItems.Add(new ToolStripMenuItem("&Salir", null, (_, _) => Close()));
menu.Items.Add(archivo);

var barra = new ToolStrip();
barra.Items.Add(new ToolStripButton("Nuevo", null, (_, _) => Nuevo()) { ToolTipText = "Nuevo
(Ctrl+N)" });
```

> Principio de diseño: toda acción de la barra de herramientas debe existir también en el menú. Así el usuario que no usa ratón puede llegar a ella con el teclado (accesibilidad).

5. Cajas de diálogo

Una caja de diálogo es una ventana secundaria que solicita información o confirma una acción. Es modal cuando bloquea la ventana principal hasta que se cierra (ShowDialog) y no modal cuando permite seguir trabajando (Show).

Diálogo	Para qué	Resultado
MessageBox.Show	Informar, advertir, confirmar	DialogResult.Yes / No / OK / Cancel
OpenFileDialog	Elegir un archivo para abrir	FileName
SaveFileDialog	Elegir dónde guardar	FileName
FontDialog / ColorDialog	Elegir fuente o color	Font / Color
Formulario propio con ShowDialog	Capturar datos (FrmProducto)	DialogResult + propiedades del formulario

```
var p = new Producto();
using var dlg = new FrmProducto(p);          // using libera la ventana al terminar
if (dlg.ShowDialog(this) == DialogResult.OK)  // modal: espera la respuesta
{
   productos.Add(p);
}

var r = MessageBox.Show(this, $"¿Eliminar «{p.Nombre}»?", "Confirmar",
   MessageBoxButtons.YesNo, MessageBoxIcon.Question, MessageBoxDefaultButton.Button2);
if (r == DialogResult.Yes) productos.Remove(p);
```

En el formulario propio, AcceptButton hace que Enter equivalga a Guardar y CancelButton que Esc equivalga a Cancelar. El botón Cancelar con DialogResult = Cancel cierra el diálogo automáticamente.

6. Integración: AgroStock v1

El código completo de AgroStock v1 está en la carpeta de recursos del Encuentro 1 (sitio del curso). Estructura del proyecto:

```
AgroStock.E01/
├── AgroStock.E01.csproj
├── Program.cs              ← punto de entrada
├── Models/Producto.cs      ← clase del dominio (POO)
└── Forms/
    ├── FrmPrincipal.cs     ← menú, barras, DataGridView, búsqueda, exportar CSV
    └── FrmProducto.cs      ← diálogo modal con validación
```

Los datos se guardan en una BindingList<Producto>: una lista que avisa a la cuadrícula cada vez que se agrega, cambia o elimina un elemento. En el Encuentro 2 la reemplazaremos por una base de datos SQLite sin cambiar la interfaz.

Ejercitación

Resuelve de forma individual. Sube capturas y el código al aula virtual (tarea «Ejercitación E1»).

Nivel 1 – Básico

1. Crea un proyecto Windows Forms «Saludo» con un TextBox, un Button y un Label. Al pulsar el botón, el Label muestra «Bienvenido, <nombre>». Si el nombre está vacío, muestra un MessageBox de advertencia.
2. Contador de existencias: dos botones (+ y −) modifican un número en un Label. El botón − se deshabilita cuando el valor llega a cero (propiedad Enabled).

Nivel 2 – Intermedio

1. Calculadora de fertilizante: el usuario ingresa el área en manzanas (NumericUpDown) y la dosis en quintales por manzana; al cambiar cualquier valor (ValueChanged) se muestra el total de quintales y su costo usando un precio fijo.
2. Agrega a tu calculadora un MenuStrip con Archivo → Salir (con confirmación en FormClosing) y Ayuda → Acerca de.

Nivel 3 – Avanzado (opcional)

1. En AgroStock v1, agrega un ContextMenuStrip a la cuadrícula con Editar y Eliminar.
2. Colorea en naranja las filas con existencia menor que 10 usando el evento CellFormatting del DataGridView.

III. Cierre y consolidación del aprendizaje

Síntesis

- Una aplicación GUI es dirigida por eventos: muestra la ventana y reacciona a las acciones del usuario.
- Cada formulario es una clase; cada control es un objeto con propiedades, métodos y eventos.
- Elegir el control adecuado (ComboBox, NumericUpDown, DateTimePicker) previene errores de captura.
- Menús, barras y atajos hacen la aplicación rápida y accesible; los diálogos confirman y capturan datos.

Aplicación (trabajo interencuentro – BOA 1)

La aplicación es el trabajo interencuentro que se retoma en la BOA 1. Debe realizarse en parejas y entregarse antes del Encuentro 2:

1. Completar AgroStock v1 según la lista de cotejo (menú, barras, formulario modal, validación, exportación CSV).
2. Investigación de campo: visitar o entrevistar un negocio local e identificar 3 procesos que AgroStock podría mejorar (una página, con evidencia).
3. Subir el proyecto (sin carpetas bin y obj) o el enlace del repositorio GitHub al aula virtual.

Complementación

- Microsoft Learn: «Tutorial: Creación de una aplicación de Windows Forms con .NET» – https://learn.microsoft.com/es-es/dotnet/desktop/winforms/get-started/create-app-visual-studio
- Microsoft Learn: «Información general sobre eventos (Windows Forms .NET)» – https://learn.microsoft.com/es-es/dotnet/desktop/winforms/forms/events
- Reflexión: ¿cómo puede una aplicación sencilla reducir el uso de papel y los viajes innecesarios de los productores? Escribe tres ideas (eje Madre Tierra).

Autoevaluación

Responde sin mirar el folleto y luego verifica con la clave al final. También puedes hacerla en línea en el sitio del curso.

1. ¿Qué hace Application.Run(new FrmPrincipal())?
   a) Compila el proyecto
   b) Muestra el formulario e inicia el bucle de mensajes
   c) Crea la base de datos
   d) Cierra la aplicación
2. ¿Qué control es el más adecuado para capturar la categoría de un producto de una lista cerrada?
   a) TextBox
   b) Label
   c) ComboBox con DropDownList
   d) Button
3. En un manejador KeyPress, ¿qué logra e.Handled = true?
   a) Cierra el formulario
   b) Descarta la tecla pulsada
   c) Convierte la tecla a mayúscula
   d) Guarda el producto
4. ¿Qué operador suscribe un método a un evento en C#?
   a) =
   b) ==
   c) +=
   d) =>
5. ¿Cuál es la diferencia entre ShowDialog() y Show()?
   a) No hay diferencia
   b) ShowDialog abre una ventana modal y devuelve DialogResult

c) Show es más rápido
d) ShowDialog solo sirve para MessageBox

6. ¿Qué significa el & en el texto "&Archivo" de un menú?

a) Es un error
b) Crea la tecla de acceso Alt+A
c) Pone la palabra en negrita
d) Agrega un separador

7. ¿En qué evento se puede cancelar el cierre de una ventana?

a) Load
b) Click
c) FormClosing
d) TextChanged

Lista de verificación: marca lo que ya lograste.

- ☐ Instalé Visual Studio 2022 con «Desarrollo de escritorio de .NET»
- ☐ Creé y ejecuté un proyecto Windows Forms
- ☐ Diseñé el formulario de producto con controles adecuados
- ☐ Programé al menos 5 eventos distintos
- ☐ Agregué MenuStrip, ToolStrip y StatusStrip
- ☐ Usé MessageBox, SaveFileDialog y un formulario modal propio
- ☐ Subí la Ejercitación E1
- ☐ Entregué AgroStock v1 y el informe de campo

Glosario

Término	Definición
GUI	Interfaz gráfica de usuario: ventanas, botones y menús en lugar de comandos de texto.
Control	Componente visual reutilizable (botón, cuadro de texto) con propiedades, métodos y eventos.
Evento	Aviso que emite un objeto cuando algo ocurre; otros objetos se suscriben para reaccionar.
Manejador de eventos	Método que se ejecuta cuando ocurre un evento; recibe sender y EventArgs.
Formulario modal	Ventana que bloquea a su ventana dueña hasta cerrarse (ShowDialog).
DialogResult	Valor que indica cómo se cerró un diálogo (OK, Cancel, Yes, No).
Dock / Anchor	Propiedades que controlan cómo un control se acomoda al cambiar el tamaño de la ventana.
BindingList	Lista que notifica sus cambios a los controles enlazados (DataGridView).

Referencias

Bibliografía del programa:

- Ceballos Sierra, F. J. Enciclopedia de Microsoft Visual C#. Editorial RA-MA.
- Deitel, H. M. y Deitel, P. J. C# Cómo programar. Editorial Pearson, Prentice Hall.

Documentación complementaria (propuesta por el docente):

- Microsoft (2024). Guía de escritorio (Windows Forms .NET). Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/
- Microsoft (2024). Microsoft.Data.Sqlite: información general. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/data/sqlite/
- Microsoft (2024). Subprocesamiento administrado y programación asincrónica. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/threading/
- Microsoft (2024). Entity Framework Core. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/
- Microsoft (2024). Realización de solicitudes HTTP con la clase HttpClient. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/fundamentals/networking/http/httpclient
- SQLite Consortium (2024). SQL As Understood By SQLite. https://www.sqlite.org/lang.html
- QuestPDF (2024). Documentación. https://www.questpdf.com/
- ClosedXML (2024). Documentación. https://docs.closedxml.io/

Clave de la autoevaluación

- 1) b – Application.Run muestra la ventana principal e inicia el bucle que convierte las acciones del usuario en eventos.
- 2) c – Un ComboBox con DropDownStyle = DropDownList impide escribir valores fuera de la lista.
- 3) b – Marcar el evento como manejado evita que el carácter llegue al TextBox.
- 4) c – btnGuardar.Click += BtnGuardar_Click; agrega el manejador a la lista de suscriptores.
- 5) b – ShowDialog bloquea la ventana dueña hasta que el diálogo se cierra y devuelve cómo se cerró.
- 6) b – La letra que sigue a & queda subrayada y se activa con Alt.
- 7) c – FormClosingEventArgs tiene la propiedad Cancel; si se pone en true la ventana no se cierra.

Nota: material elaborado con apoyo de inteligencia artificial para la organización y redacción inicial; revisado por el docente.